

Cloudflare Tunnel

The Complete Guide (cloudflared) — Zero to Hero

Install cloudflared, run a quick tunnel with no account needed, set up a permanent named tunnel tied to your own domain, forward raw TCP services, run it as a background service, and follow good security practices.

Table of Contents

1. What is Cloudflare Tunnel?
2. Installing cloudflared (Windows / macOS / Linux)
3. Quick Tunnel — No Domain or Login Required
4. The Permanent Route — Logging Into Your Cloudflare Account
5. Creating a Named Tunnel
6. Routing a Domain to Your Tunnel (DNS)
7. The config.yml File and Ingress Rules
8. Running the Tunnel & Installing It as a Service
9. Raw TCP Tunnels (SSH, custom sockets)
10. Useful Commands Cheat Sheet
11. Security Best Practices
12. Troubleshooting

1. What is Cloudflare Tunnel?

Cloudflare Tunnel (its CLI tool is called `cloudflared`) is a secure way to connect a local service — a web server, an API, an SSH server, or any other TCP service — to the internet through Cloudflare's network, without opening any ports on your router or firewall.

How it differs from something like ngrok:

- You can attach a permanent, free subdomain on your own domain (once its DNS is on Cloudflare)
- A tunnel is defined as a "Named Tunnel" with a fixed ID, so it keeps the same address every time (unlike random links)
- It's completely free, even for long-term or light production use
- It can forward both HTTP/HTTPS traffic and raw TCP traffic

Like ngrok, the `cloudflared` agent on your machine opens an outbound connection to Cloudflare's servers; inbound traffic to your domain reaches your local service through that same connection.

2. Installing cloudflared

Windows

Download the installer from:

```
https://github.com/cloudflare/cloudflared/releases/latest
```

Or with winget:

```
winget install --id Cloudflare.cloudflared
```

macOS

```
brew install cloudflared
```

Linux (Debian/Ubuntu)

```
sudo mkdir -p --mode=0755 /usr/share/keyrings
curl -fsSL https://pkg.cloudflare.com/cloudflare-main.gpg \
| sudo tee /usr/share/keyrings/cloudflare-main.gpg >/dev/null

echo 'deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg]
https://pkg.cloudflare.com/cloudflared any main' \
| sudo tee /etc/apt/sources.list.d/cloudflared.list

sudo apt update && sudo apt install cloudflared
```

Linux (direct binary — works on any distro)

```
curl -L --output cloudflared \
https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64
chmod +x cloudflared
sudo mv cloudflared /usr/local/bin/
```

Verify the install:

```
cloudflared --version
```

3. Quick Tunnel — No Domain or Login Required

If you just want to quickly expose a local service temporarily and don't have a domain or Cloudflare account, this is the simplest route. Say your service runs on port 1080:

```
cloudflared tunnel --url tcp://localhost:1080
```

Or for a web server on port 3000:

```
cloudflared tunnel --url http://localhost:3000
```

The output gives you a random link under the `trycloudflare.com` domain, e.g.:

```
https://random-words-here.trycloudflare.com
```

This "Quick Tunnel" mode is temporary — no login or account is required, but every time you run it you get a new random link, and it isn't suited for long-term or production use. For a permanent address, follow the sections below.

4. The Permanent Route — Logging Into Your Cloudflare Account

To build a permanent tunnel with a fixed address, you first need a Cloudflare account (free) and a domain whose DNS you've moved to Cloudflare.

- 1 If you don't have a domain, or don't want to attach one, you can skip this section and just use the Quick Tunnel from section 3.
- 2 Log into your account from the terminal:

```
cloudflared tunnel login
```

This opens a link in your browser; log in and pick one of the domains under your account. After confirming, a certificate file (typically `cert.pem`) is saved under `~/.cloudflared/`, which authenticates you when creating tunnels.

5. Creating a Named Tunnel

After logging in, create a tunnel with a name of your choice:

```
cloudflared tunnel create mytunnel
```

This generates a unique UUID for your tunnel and saves a credentials file (`<tunnel-id>.json`) under `~/.cloudflared/`. To see the tunnels you've created:

```
cloudflared tunnel list
```

6. Routing a Domain to Your Tunnel (DNS)

Now attach a subdomain of your domain to this tunnel:

```
cloudflared tunnel route dns mytunnel proxy.yourdomain.com
```

This automatically creates a CNAME record in your domain's DNS (managed by Cloudflare) pointing at your tunnel. From now on, `proxy.yourdomain.com` always routes to this same tunnel — permanent and fixed.

7. The config.yml File and Ingress Rules

Create a config file (typically at `~/.cloudflared/config.yml`) that specifies which local service each subdomain should forward to:

```
tunnel: mytunnel
credentials-file: /home/user/.cloudflared/<tunnel-id>.json

ingress:
- hostname: proxy.yourdomain.com
  service: tcp://localhost:1080
- hostname: app.yourdomain.com
  service: http://localhost:3000
- service: http_status:404
```

Key points about ingress rules:

- Each hostname must already have been routed to this tunnel using the command from section 6
- You can define several different services (one TCP, one HTTP) under a single tunnel
- The last rule must always be a catch-all with no hostname (typically `http_status:404`) — this is mandatory and must be the final item in the list

8. Running the Tunnel & Installing It as a Service

To run the tunnel manually using the config file you created:

```
cloudflared tunnel run mytunnel
```

If your config file is somewhere else or has a different name, point to it explicitly:

```
cloudflared tunnel --config /path/to/config.yml run mytunnel
```

Running as a system service (so it survives a reboot)

On Linux (systemd):

```
sudo cloudflared service install
sudo systemctl start cloudflared
sudo systemctl enable cloudflared
```

On Windows:

```
cloudflared service install
```

These commands install cloudflared as a background service that automatically runs with your default config.

9. Raw TCP Tunnels (SSH, custom sockets)

As shown in section 7, by setting `service: tcp://localhost:PORT` in your ingress rules, you can forward any TCP service (not just HTTP) — for example, the client/server script that used port 1080, or an SSH server.

On the client side (whoever needs to connect to this service), reaching it through Cloudflare requires the `cloudflared access` tool, which creates a local TCP proxy:

```
cloudflared access tcp --hostname proxy.yourdomain.com --url localhost:1080
```

This opens a local port (1080) on the client machine that tunnels all its traffic through Cloudflare to the actual tunnel server. That means in your client script, you just connect to `localhost:1080`; `cloudflared access` handles the rest of the route.

For private (non-public) TCP tunnels, Cloudflare Access and authentication policies are typically layered on top so only authorized users can connect — this is more advanced and configured in the Cloudflare Zero Trust dashboard.

10. Useful Commands Cheat Sheet

Command	What it does
<code>cloudflared tunnel --url http://localhost:1080</code>	Temporary Quick Tunnel, no login

<code>cloudflared tunnel login</code>	Log into your Cloudflare account
<code>cloudflared tunnel create NAME</code>	Create a new permanent tunnel
<code>cloudflared tunnel list</code>	List tunnels you've created
<code>cloudflared tunnel route dns NAME hostname</code>	Attach a subdomain to a tunnel
<code>cloudflared tunnel run NAME</code>	Run a tunnel based on its config file
<code>cloudflared tunnel delete NAME</code>	Delete a tunnel
<code>cloudflared service install</code>	Install cloudflared as a permanent system service
<code>cloudflared access tcp --hostname H --url U</code>	Create local TCP proxy on the client side

11. Security Best Practices

- Never share your `cert.pem` or `<tunnel-id>.json` files anywhere — these authenticate your tunnel and account identity.
- For sensitive services, put Cloudflare Access (authentication before traffic reaches your local service) in front of it, not just an open public tunnel.
- Only define the ports you actually need in your ingress rules — don't expose extra services.
- Stop the tunnel when you're done, or if you installed it as a permanent service, run `systemctl stop cloudflared` when needed.

Any hostname you define and route in your ingress rules is reachable from anywhere on the internet as long as the tunnel is running (unless you put Cloudflare Access or an authentication policy behind it). Don't expose services with no password or default settings this way.

12. Troubleshooting

Problem	Likely fix
"tunnel credentials file not found"	Check the credentials-file path in config.yml against the actual JSON file location
Subdomain won't load / error 1033	Make sure you ran <code>tunnel route dns</code> for that exact hostname and the domain's DNS is pointing to the tunnel
"failed to login" during <code>cloudflared tunnel login</code>	Check your default browser; on a headless server, manually open the printed URL
Service doesn't come back up after reboot	Make sure you ran <code>cloudflared service install</code> and the service is enabled
Connection refused on the hostname	Verify your local service is actually running on the port you specified in ingress rules

Official docs: <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>